E1 294O: EDGE AND CLOUD SYSTEMS FOR MACHINE LEARNING ALGORITHMS (JAN'26)

Assignment 4

Author
Ritik Agarwal
MTech DSBA, IISc
27/03/2026

Contents

1 Problem 1 - Let's do some math

1.1 KV Cache Calculation

Given Input:

- Model Specification
  - Number of Layers - L
  - Number of Query heads - n_{q_heads}
  - Number of KV heads - n_{kv_heads}
  - Embedding Dimension (h_1) - d_{model}
  - Inner Dimension (h_2) - d_{ff}
  - Vocab Size - V
- Device Specification
  - GPU Memory - mem_{gpu}
- Input Specification
  - Context Length (s) - seq_len
  - Weights bytes - w_{bytes}
  - KV cache bytes - kv_{bytes}

Calculation Formulae

We follow the steps below to calculate the parameters for each component of the model:

- **Head Dimension:** The number of columns for each attention head is calculated by dividing the model's embedding dimension by the number of query heads. We don't use the number of KV heads because in some models, where the number of KV heads is less than the query heads, the KV heads are shared across multiple query heads.

$$d_{head} = \frac{d_{model}}{n_{q\_heads}}$$

- **Attention Parameters (per layer):** This calculates the number of elements in the Query matrices, the Key and Value matrices, and the output matrix after the attention block.

$$P_{attn} = (n_{q\_heads} \cdot d_{model} \cdot d_{head}) + (2 \cdot n_{kv\_heads} \cdot d_{model} \cdot d_{head}) + d_{model}^2$$

- **Feed-Forward Parameters (per layer):** The up projection and down projection contain 1 matrix each, both being transposes of each other.

$$P_{ffn} = (d_{model} \cdot d_{ff}) + (d_{ff} \cdot d_{model}) = 2 \cdot d_{model} \cdot d_{ff}$$

- **Vocabulary Embeddings:** At the input and output stages of a transformer block, the model uses the full vocabulary embeddings. These are used to map the input prompt to Token IDs and the model output logits back to Token IDs, respectively.

$$P_{v\_embed} = 2 \cdot V \cdot d_{model}$$

- **Total Model Parameters:** Sum up all the parameters from the vocabulary embeddings, attention heads for all layers, and feed-forward parameters for all layers.

$$Model_{params} = (2 \cdot V \cdot d_{model} + L \cdot (P_{attn} + P_{ffn}))$$

- **Total Model Weights Memory:** The total parameters across all L layers are multiplied by the memory of each parameter.

$$Mem_{weights} = Model_{params} \cdot w_{bytes}$$

- **KV Cache Memory (per request):** The memory required to store the Key and Value matrices across all layers for a given input sequence length (s).

$$Mem_{kvc} = 2 \cdot n_{kv\_heads} \cdot s \cdot d_{head} \cdot L \cdot kv_{bytes}$$

- **Maximum Batch Size:** The theoretical maximum batch size is calculated by taking the available GPU memory (in bytes), subtracting the model weights required, and dividing by the KV cache memory required for a single request.

$$BS_{max} = \left\lfloor \frac{(mem_{gpu} \cdot 1024^3) - Mem_{weights}}{Mem_{kvc}} \right\rfloor$$

1.1.1 Llama-3-8B Calculation

For a Llama-3-8B model, one change is required to the Feed-Forward Parameter calculation. This model uses the SwiGLU activation function in the FF block, which requires an additional Gate Matrix of the same size as the Up projection matrix. Thus, the parameters in the feed-forward block are calculated as:

$$P_{ffn} = 2 \cdot (d_{model} \cdot d_{ff}) + (d_{ff} \cdot d_{model}) = 3 \cdot d_{model} \cdot d_{ff}$$

Additionally, the given Llama-3-8B spec:

- Number of Layers (L) - 32
- Number of Query heads (n_q) - 32
- Number of KV heads (n_{kv}) - 8
- Embedding Dimension (h_1) - 4096
- Inner Dimension (h_2) - 14336
- Vocab Size (V) - 128256

- GPU Memory - 80 GB
- Input Sequence Length - s (unknown)
- Weight Precision - w_{bytes} (unknown)
- KV Cache Precision - kv_{bytes} (unknown)

With the modified P_{ffn}, and the formulae and spec mentioned above, we get the following results for Llama-3-8B:

Metric	**Calculation / Value**
Total Model Parameters	$P_{total} = 8,029,995,008$
Total Parameter Memory (GB)	$Mem_{weights} = \frac{P_{total} \cdot w_{bytes}}{1024^3}$
KV Cache Per Request (GB)	$Mem_{kvc} = \frac{65,536 \cdot s \cdot kv_{bytes}}{1024^3}$
Max Batch Size Possible	$BS_{max} = \left\lfloor \frac{80 - Mem_{weights}}{Mem_{kvc}} \right\rfloor$

Table 1: Model Memory and Batch Size Calculations for Llama-3-8B

1.2 KV Cache with TP

When the model and the KV Cache are split across multiple GPUs with tensor parallel dimension X, the effective memory available increases to $mem_{gpu} \times X$. In this case, most variables remain the same. But we can calculate all the variables as follows:

- **Head Dimension:**

$$d_{head} = \frac{d_{model}}{n_{q\_heads}}$$

- **Attention Parameters (per layer):**

$$P_{attn} = (n_{q\_heads} \cdot d_{model} \cdot d_{head}) + (2 \cdot n_{kv\_heads} \cdot d_{model} \cdot d_{head}) + d_{model}^2$$

- **Feed-Forward Parameters (per layer):**

$$P_{ffn} = (d_{model} \cdot d_{ff}) + (d_{ff} \cdot d_{model}) = 2 \cdot d_{model} \cdot d_{ff}$$

- **Vocabulary Embeddings:**

$$P_{v\_embed} = 2 \cdot V \cdot d_{model}$$

- **Total Model Parameters:**

$$Model_{params} = (2 \cdot V \cdot d_{model} + L \cdot (P_{attn} + P_{ffn}))$$

- **Total Model Weights Memory (per GPU):** The total parameters across all L layers are multiplied by the memory of each parameter and divided by TP dim X.

$$Mem_{weights\_per\_gpu} = \frac{Model_{params} \cdot w_{bytes}}{X}$$

- **KV Cache Memory (per request, per GPU):** The memory required to store the Key and Value matrices across all layers for a given input sequence length (s), divided across the available GPUs.

$$Mem_{kvc\_per\_gpu} = \frac{2 \cdot n_{kv\_heads} \cdot s \cdot d_{head} \cdot L \cdot kv_{bytes}}{X}$$

- **Maximum Batch Size:** Since we have already calculated per-GPU memory requirements, we can use the single-GPU memory to calculate the theoretical maximum batch size by taking the available GPU memory (in bytes), subtracting the model weights requirement per GPU, and dividing by the KV cache memory required for a single request (per GPU).

$$BS_{max} = \left\lfloor \frac{(mem_{gpu} \cdot 1024^3) - Mem_{weights\_per\_gpu}}{Mem_{kvc\_per\_gpu}} \right\rfloor$$

- **Straight Forward Maximum Batch Size:** We could also calculate the maximum batch size by multiplying the available GPU memory by TP dimension X. In this case, we directly use the total memory required by the model and the KV Cache memory required by a single request:

$$BS_{max} = \left\lfloor \frac{(mem_{gpu} \cdot 1024^3 \cdot X) - Mem_{weights}}{Mem_{kvc}} \right\rfloor$$

1.2.1 KV Cache with TP for Llama-3-8B

Similar to the subsection before, we use the modified formula for the feed-forward network in Llama-3-8B due to the SwiGLU activation function:

$$P_{ffn} = 2 \cdot (d_{model} \cdot d_{ff}) + (d_{ff} \cdot d_{model}) = 3 \cdot d_{model} \cdot d_{ff}$$

Metric	**Calculation / Value**
Total Model Parameters	$P_{total} = 8,029,995,008$
Total Parameter Memory (GB, per GPU)	$Mem_{weights\_per\_gpu} = \frac{P_{total} \cdot w_{bytes}}{1024^3 \cdot X}$
KV Cache Per Request (GB, per GPU)	$Mem_{kvc\_per\_gpu} = \frac{65,536 \cdot s \cdot kv_{bytes}}{1024^3 \cdot X}$
Max Batch Size Possible for TP X	$BS_{max} = \left\lfloor \frac{80 - Mem_{weights\_per\_gpu}}{Mem_{kvc\_per\_gpu}} \right\rfloor$

Table 2: Model Memory and Batch Size Calculations for Llama-3-8B with TP X

1.3 Calculate Arithmetic Intensity for Attention

Given Dimensions for $Q, K, V = N \times d$
Context length = N
Embedding dimension = d
Bytes per Element = w_{bytes}

Matrix Multiplication FLOPs Formula
For any two matrices $A_{m \times n} \times B_{n \times p}$:

- Total number of Multiplications = $m \cdot n \cdot p$
- Total number of Additions = $m \cdot p \cdot (n-1)$
- Total Floating Point Operations = $m \cdot n \cdot p + m \cdot p \cdot (n-1) = m \cdot p \cdot (2n-1)$

Assumptions for our calculations:

- While the industry standard for FLOPs = $2 \cdot m \cdot n \cdot p$, we are using the exact formula (as shown above) in our calculations.
- We ignore the FLOPs for $softmax$ since its computational requirement is negligible compared to the large matrix multiplications.

Step-wise Calculation of FLOPs and Memory Access for Attention

1. Load $Q_{N \times d}$ and $K_{N \times d}$ by blocks from HBM.
   - Memory Access = $2 \cdot N \cdot d \cdot w_{bytes}$
2. Compute $S_{N \times N} = Q_{N \times d} \cdot K^T_{d \times N}$.
   - FLOPs = $N^2 \cdot (2d-1)$
3. Write $S_{N \times N}$ to HBM.
   - Memory Access = $N^2 \cdot w_{bytes}$
4. Read $S_{N \times N}$ from HBM.
   - Memory Access = $N^2 \cdot w_{bytes}$
5. Write $P_{N \times N}$ to HBM.
   - Memory Access = $N^2 \cdot w_{bytes}$
6. Load $P_{N \times N}$ and $V_{N \times d}$ by blocks from HBM.
   - Memory Access = $N^2 \cdot w_{bytes} + N \cdot d \cdot w_{bytes}$
7. Compute $O_{N \times d} = P_{N \times N} \cdot V_{N \times d}$.
   - FLOPs = $N \cdot d \cdot (2N-1)$
8. Write $O_{N \times d}$ to HBM.
   - Memory Access = $N \cdot d \cdot w_{bytes}$

Final Aggregated Totals

- **Total FLOPs (N_{flops}):**
  Summing the FLOPs from steps 2 and 8:

$$N_{flops} = N^2(2d-1) + Nd(2N-1) = 4N^2d - N^2 - Nd$$

- **Total Memory Access (Mem_{bytes}):**
  Summing the memory access across all steps:

$$Mem_{bytes} = (2Nd+N^2+N^2+N^2+N^2+Nd+Nd)\cdot w_{bytes} = (4N^2+4Nd)\cdot w_{bytes} = 4N(N+d)\cdot w_{bytes}$$

- **Arithmetic Intensity (AI):**

$$AI = \frac{N_{flops}}{Mem_{bytes}} = \frac{4N^2d - N^2 - Nd}{4N(N+d)\cdot w_{bytes}} = \frac{4Nd - N - d}{4(N+d)\cdot w_{bytes}}$$

Metric	**Calculated Value**
Total FLOPs (N_{flops})	$4N^2d - N^2 - Nd$
Total Memory Access ($bytes$)	$4N(N+d)\cdot w_{bytes}$
Arithmetic Intensity ($FLOPs/bytes$)	$\frac{4Nd - N - d}{4(N+d)\cdot w_{bytes}}$

Table 3: Standard Attention Computational and Memory Requirements

1.4 Calculate Arithmetic Intensity and Plot Curves

For this section, we assume $w_{bytes} = 2$

Given:

- N = 1024
- d = 128

Arithmetic Intensity (AI):

$$AI = \frac{4Nd - N - d}{4(N+d)\cdot w_{bytes}}$$

$$AI = \frac{4\cdot 1024\cdot 128 - 1024 - 128}{4\cdot(1024+128)\cdot 2} = 56.76$$

Therefore, **Arithmetic Intensity = 56.76 FLOPs / bytes**

The graphs are plotted with $d = 128$ and $w_{bytes} = 2$. We take two scenarios under consideration:

- **Prefill:** N is varied while keeping a single batch and generating only a single token. The formula is directly used with the value of N.
- **Decode:** The batch size B is varied, with each request generating 1 token. Thus, the input is 1 token per request, but B requests are batched together. Therefore, the same formula applies, but $N = B$.

When plotting the charts with the absolute values of N and B, we experienced that the points were too concentrated, reducing graph readability. Hence, we have used a log-scale on the X-Axis for both charts, while keeping the labels as the absolute values.

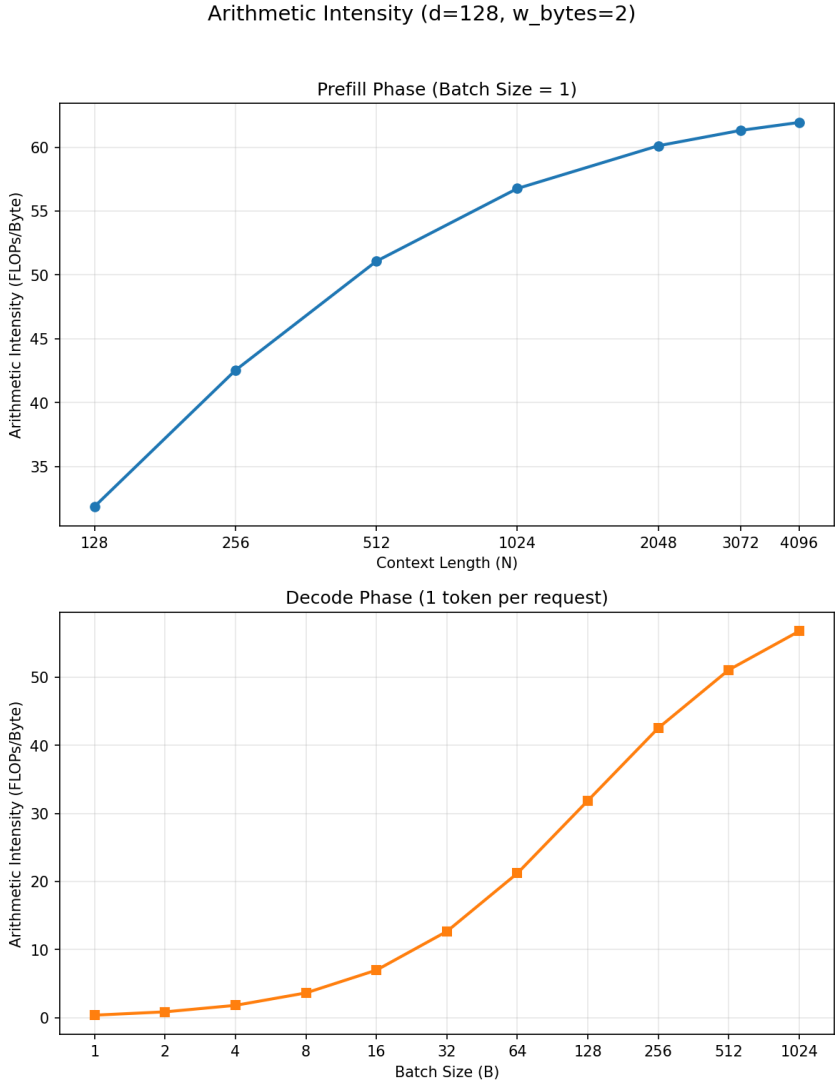


Figure 1: Arithmetic Intensity vs Context Length for Prefill and Arithmetic Intensity vs Batch Size for Decode

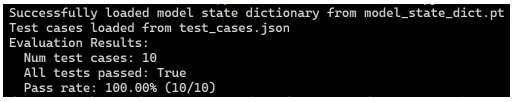
A few observations from Figure 1:

- **Compute-Bound Prefill and Memory-Bound Decode:** Considering practical prefill lengths (128 to 4096) and practical decode batch sizes (1 to 128), we see that prefill achieves high AI constantly, while decode stays much lower. Thus, prefill is primarily bound by the GPU compute units. In decode, the GPU spends significant time loading memory, making it heavily memory-bound.
- **Diminishing Returns:** The gain in AI for prefill from 128 $\rightarrow$ 256 is much higher than 1024 $\rightarrow$ 2048, resulting in lower returns as context length is increased. While AI increases at higher batch sizes for decode, practical batches span from 128 to 256 requests. From the figure, returns begin to diminish at 512 requests.
- **Mathematical Formula :** While the formula remains the same, the characteristics of prefill and decode produce vastly different curves.

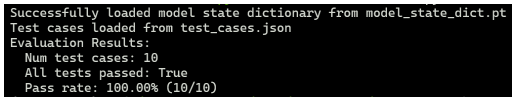
2 Problem 2 - Transformer Implementation with KV Caching

All the TODO sections in the given `transform_skeleton.py` have been implemented. Both prefill and generate are implemented completely with float32 precision.

The terminal outputs for both evaluation modes are shown in Figure 2. Specifically, the output of `--mode evaluate` is presented in Figure 2a, and the output of `--mode kv_evaluate` is presented in Figure 2b. While the tolerance requirement has been updated to 2 decimal places, below are the results for both 2 and 5 decimal places.

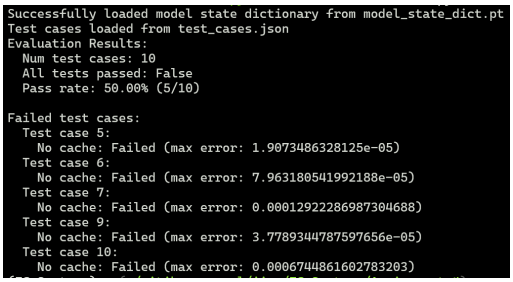


(a) Without KV Cache (`evaluate`)

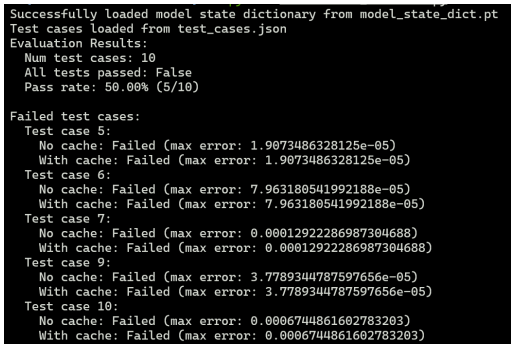


(b) With KV Cache (`kv_evaluate`)

Figure 2: Terminal output logs for both evaluation modes for 2 decimal places.



(a) Without KV Cache (`evaluate`)



(b) With KV Cache (`kv_evaluate`)

Figure 3: Terminal output logs for both evaluation modes for 5 decimal places.

For the benchmarking graph, the following parameters were used:

- Input Sequence Lengths: $[10, 50, 100, 500, 1000]$
- Tokens Generated: 40
- Modes Benchmarked: With and Without KV Cache

Figure 4 shows the Latency vs. Sequence Length plots. A few observations:

- For very short sequences, the KV Cache memory transfer cost is more than re-computing the KV matrices, resulting in lower latency for the run without caching.
- The latency with KV Cache becomes nearly constant, even when sequence length is increased from 500 to 1000. Conversely, the latency without KV Cache strictly increases.
- Dropping the CPU caches between each run helped produce more stable benchmarks. Otherwise, the average gains over multiple runs with KV Cache were lesser than expected.

Table 4 summarizes the gains with KV Cache.

Table 4: Summary of speedup gains using KV Cache

Sequence Length	**No Cache (s)**	**KV Cache (s)**	**Speedup**
10	0.0236	0.0415	0.57x
50	0.0485	0.0431	1.12x
100	0.1613	0.0324	4.97x
500	0.2612	0.0339	7.70x
1000	1.0383	0.1152	9.01x

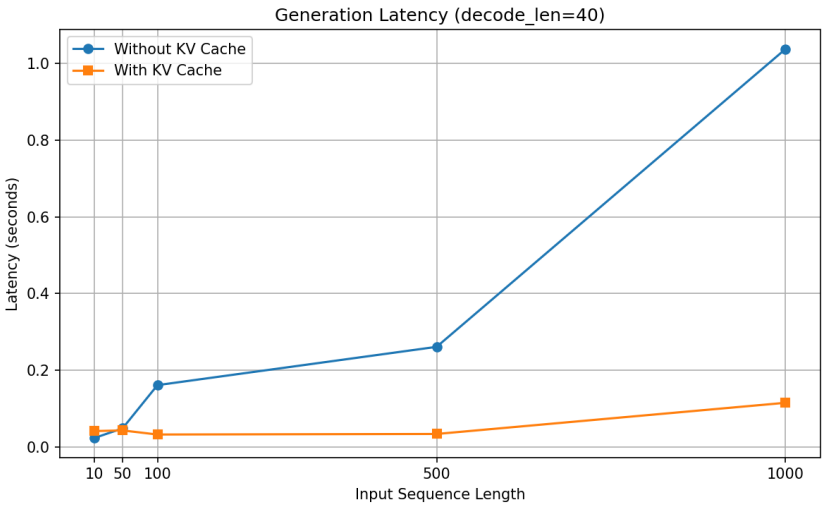


Figure 4: Latency comparison with and without KV Cache across various sequence lengths.